Open Object
Close Object
Break Object
Cook Object
Slice Object
Toggle Object
Dirty Object
Fill Object with Liquid
Empty Liquid from Object
Use Up Object

Open Object

`OpenObject` attempts to open an Openable object to a specified amount, parameterized by `openness`. The full list of openable object types can be filtered on the Object Types page.

Warning

An object can fail to open if it hits another object as it is opening. In this case the action will fail and the target object will reset to the position it was last in.

```
event = controller.step(
    action="OpenObject",
    objectId="Book|0.25|-0.27|0.95",
    openness=1,
    forceAction=False
)
```

Open Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

openness: float = 1
The proportion regarding how far the object should open. Valid values are in $[0:1]$, where the value 0 corresponds to completely closed, 1 corresponds to completely open, and 0.5 corresponds to halfway open, for instance.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to the state of its openness.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "openable": True,
    "openness": 0.75,
    "isOpen": True,

    {...}
}
```

Open Object Response

openable: bool
Can this object be opened?

openness: float
The proportion that the object is open, linearly scaled to the range $[0:1]$. For instance, if an object is a quarter of the way open, then its openness would be 0.25.

isOpen: bool
`False` when the object's openness is 0, `True` otherwise.

Close Object

`CloseObject` is syntactic sugar for `OpenObject` with an `openness` of 0.

Warning

An object can fail to close if it hits another object as it is opening. In this case the action will fail and the target object will reset to the position it was last in.

```
controller.step(
    action="CloseObject",
    objectId="Book|0.25|-0.27|0.95",
    forceAction=False
)
```

Close Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Break Object

`BreakObject` attempts to break a breakable object. If successful, the object appears visibly different. For instance, breakable objects may shatter completely into pieces or have their screens cracked. The full list of breakable object types can be filtered on the Object Types page. Broken objects cannot be unbroken (until the scene has reset).

```
event = controller.step(
    action="BreakObject",
    objectId="Vase|0.25|-0.27|0.95",
    forceAction=False
)
```

Break Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is broken.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "breakable": True,
    "isBroken": False,

    {...}
}
```

Break Object Response

breakable: bool
Can this object break?

isBroken: bool
Is this object currently broken?

Cook Object

`CookObject` attempts to switch an object to its cooked state. Objects cannot be uncooked (unless the scene is reset). The full list of cookable object types can be filtered on the Object Types page.

```
event = controller.step(
    action="CookObject",
    objectId="Egg|0.25|-0.27|0.95",
    forceAction=False
)
```

Cook Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is cooked.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "cookable": True,
    "isCooked": False,

    {...}
}
```

Cook Object Response

cookable: bool
Can this object be cooked?

isCooked: bool
Is this object currently cooked?

Slice Object

`SliceObject` attempts to slice an object.

If an object, such as an Apple, is successfully sliced, there may include several new AppleSliced objects in the scene and metadata, one for each slice. Other objects, like Egg, may only break after being sliced. Sliced objects cannot be unsliced (unless the scene is reset).

The full list of sliceable objects can be filtered on the Object Types page.

```
event = controller.step(
    action="SliceObject",
    objectId="Potato|0.25|-0.27|0.95",
    forceAction=False
)
```

Slice Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is sliced.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "sliceable": True,
    "isSliced": False,

    {...}
}
```

Slice Object Response

sliceable: bool
Can this object be sliced?

isSliced: bool
Is this object currently sliced?

Toggle Object

`ToggleObjectOn` and `ToggleObjectOff` toggles an object between on and off states, respectively. Examples include Lamps, Light Switches, Stove Knobs, and Laptops. The full list of toggleable objects can be filtered on the Object Types page.

Toggle On

```
event = controller.step(
    action="ToggleObjectOn",
    objectId="LightSwitch|0.25|-0.27|0.95",
    forceAction=False
)
```

Toggle Off

```
event = controller.step(
    action="ToggleObjectOff",
    objectId="LightSwitch|0.25|-0.27|0.95",
    forceAction=False
)
```

Toggle Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is toggled.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "toggleable": True,
    "isToggled": True,

    {...}
}
```

Toggle Object Response

toggleable: bool
Can this object be toggled?

isToggled: bool
Is this object currently toggled on?

Dirty Object

`DirtyObject` attempts to make an object look dirty. `CleanObject` can then be used to make the object look clean again. The full list of dirtyable object types can be filtered on the Object Types page.

Dirty Object

```
event = controller.step(
    action="DirtyObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Clean Object

```
event = controller.step(
    action="CleanObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Dirty Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is toggled.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "toggleable": True,
    "isToggled": True,

    {...}
}
```

Dirty Object Response

toggleable: bool
Can this object be toggled?

isToggled: bool
Is this object currently toggled on?

Fill Object with Liquid

`FillObjectWithLiquid` attempts to fill an object with `fillLiquid`. Only compatible objects that are empty can be filled. The full list of object types that can be filled with liquid can be filtered on the Object Types page.

```
event = controller.step(
    action="FillObjectWithLiquid",
    objectId="Mug|0.25|-0.27|0.95",
    fillLiquid="coffee",
    forceAction=False
)
```

Fill Object with Liquid Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

fillLiquid: string required
The type of liquid that fills the object. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is filled with liquid.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "fillLiquid": "coffee",
    "canFillWithLiquid": True,
    "isFilledWithLiquid": True,

    {...}
}
```

Fill Object with Liquid Response

fillLiquid: Optional[str]
Which liquid is the object filled with? If the object is not filled with liquid, it will report `None`. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

canFillWithLiquid: bool
Can this object be filled with liquid?

isFilledWithLiquid: bool
Is this object currently filled with liquid?

Empty Liquid from Object

`EmptyLiquidFromObject` attempts to empty the liquid from an object if it is currently filled with liquid (i.e., `isFilledWithLiquid` is True).

Remark

If an object is rotated downward far enough, gravity may cause the object to lose its liquid automatically, causing `isFilledWithLiquid` to be `False`. Here, the liquid will not appear spilled on the ground like a puddle. Instead, it disappears.

```
event = controller.step(
    action="EmptyLiquidFromObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Empty Liquid From Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it can be emptied of liquid.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "fillLiquid": None,
    "canFillWithLiquid": True,
    "isFilledWithLiquid": False,

    {...}
}
```

Empty Liquid from Object Response

fillLiquid: Optional[str]
Which liquid is the object filled with? If the object is not filled with liquid, it will report `None`. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

canFillWithLiquid: bool
Can this object be emptied with liquid?

isFilledWithLiquid: bool
Is this object currently filled with liquid?

Use Up Object

`UseUpObject` attempts to use up parts of an object. The action works with objects like ToiletPaper, TissueBox, and PaperTowelRoll, which each has an full/empty state. The full list of object types that can be used up can be filtered on the Object Types page.

```
event = controller.step(
    action="UseUpObject",
    objectId="ToiletPaper|0.25|-0.27|0.95",
    forceAction=False
)
```

Use Up Object Parameters

objectId: string required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is used up.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "canBeUsedUp": True,
    "isUsedUp": False,

    {...}
}
```

Use Up Object Response

canBeUsedUp: bool
Can this object be used up?

isUsedUp: bool
Is this object currently used up?